

GLM-5.1 本地部署方案深度调研

葛良晨 by Claude Code

2026 年 5 月 19 日

文档版本历史

2026-05-19 初始版本，完成 GLM-5.1 模型的本地部署方案全面调研，涵盖架构分析、硬件选型、量化策略、推理框架对比与全场景成本预估。

1 GLM-5.1 模型概述

1.1 基本信息

GLM-5.1 是智谱 AI (Z.ai / THU DM) 于 2025 年中发布的新一代旗舰大语言模型，定位为“Agentic Engineering (智能体工程)”时代的基座模型。其核心定位是：**不是单纯的对话模型，而是面向长时间、多轮次、数千次工具调用的自主编程智能体。**

表 1: GLM-5.1 基本信息

项目	详情
开发者	智谱 AI (Z.ai / THU DM)
模型架构	GLM MoE + DSA (DeepSeek Sparse Attention)
总参数量	744B (7440 亿)
激活参数量	40B (400 亿)，每次推理仅激活约 5.4% 的参数
精度格式	BF16 (原始)、FP8 (官方优化版)
上下文窗口	128K tokens
开源协议	MIT (HuggingFace) / Apache 2.0 (GitHub)
技术报告	arXiv:2602.15763
模型页	huggingface.co/zai-org/GLM-5.1

1.2 架构特点：MoE + DSA

GLM-5.1 的架构标签为 `glm_moe_dsa`，融合了两项核心技术：

MoE (Mixture of Experts, 混合专家)：模型包含多个“专家”子网络，每次推理仅激活其中一部分。GLM-5.1 的 744B 总参数中仅有 40B 在单次前向传播中激活，这使得推理计算量仅与 40B 稠密模型相当，而模型的知识容量却达到 744B 级别。这是“大模型能力、小模型成本”的关键设计。

DSA (DeepSeek Sparse Attention, 深度稀疏注意力)：源自 DeepSeek 系列的稀疏注意力机制，通过智能选择哪些 Key-Value 对被保留到缓存中，大幅降低了长上下文场景下的 KV Cache 显存占用。官方表述为“largely reducing deployment cost while preserving long-context capacity”。

1.3 性能定位

GLM-5.1 在多个权威基准上达到了开源模型的最高水平，甚至在部分指标上超越了闭源商业模型。

表 2: GLM-5.1 核心基准测试成绩

基准测试	得分	排名/备注
SWE-Bench Pro（软件工程）	58.4	#1 SOTA，超越 Claude Opus 4.6
CyberGym（网络安全）	68.7	#1 SOTA
BrowseComp（网页理解）	68.0	#1 SOTA
NL2Repo（自然语言到仓库）	42.7	#2，仅次于 Claude Opus 4.6 (49.8)
Terminal-Bench 2.0	63.5	终端环境编程能力
AIME 2026（数学竞赛）	95.3	数学推理
HMMT Feb 2026（数学竞赛）	82.6	数学推理
GPQA-Diamond（研究生知识）	86.2	知识推理
HLE（难例评测）	31.0	带工具提升至 52.3
MCP-Atlas（工具调用）	71.8	多工具协同

关键差异化能力：GLM-5.1 的设计重心并非单轮对话质量，而是”the longer it runs, the better the result”——在数百轮迭代、数千次工具调用的长时间自主编程任务中，模型不会像前代那样过早陷入局部最优或”迷失方向”。这一特性使其在 SWE-Bench Pro 等需要持续优化的复杂工程任务上取得突破。

1.4 与竞品模型的参数对比

表 3: GLM-5.1 与主流开源 MoE 模型参数对比

模型	总参数	激活参数	激活比	上下文
GLM-5.1	744B	40B	5.4%	128K
GLM-5	744B	40B	5.4%	128K
GLM-4.5	355B	32B	9.0%	128K
DeepSeek-V3	671B	37B	5.5%	128K
Qwen3-235B	235B	22B	9.4%	128K
Mixtral 8×22B	141B	39B	27.7%	64K

GLM-5.1 的激活比（5.4%）在同类 MoE 模型中处于极低水平，意味着相同推理成本下可以承载更大的知识容量。但这也意味着**模型权重的静态存储需求极大**——744B 参数即使不全部激活，也必须全部加载到显存/内存中。

2 硬件需求分析

2.1 显存需求的理论计算

GLM-5.1 作为 744B 参数的 MoE 模型，其显存需求比同级别稠密模型更为复杂。以下是完整的显存占用分解。

表 4: 不同精度下 GLM-5.1 权重的显存占用

精度	每参数字节	纯权重占用	说明
BF16	2 bytes	≈1,488 GB	原始精度，不可单机承载
FP8	1 byte	≈744 GB	官方优化版，需 8×H100
INT8	1 byte	≈744 GB	与 FP8 等同
INT4	0.5 bytes	≈372 GB	GGUF Q4_K_M 典型值
NVFP4	~0.59 bytes	≈437 GB	NVIDIA FP4 专用格式
MXFP4	~0.55 bytes	≈408 GB	AMD FP4 专用格式
INT2	0.25 bytes	≈186 GB	极低精度，质量损失严重

2.1.1 权重部分的显存占用

2.1.2 完整显存账单（推理时）

推理时的总显存占用远不止权重本身。以一个实际的部署场景为例：

$$\text{VRAM}_{\text{total}} = \text{权重} + \text{KV Cache} + \text{激活值} + \text{框架开销} + \text{MoE 专家缓冲区} \quad (1)$$

以官方推荐的 8 GPU 部署、128K 上下文为例（假设使用 FP8 精度）：

- **权重**：744 GB (FP8)，均分到 8 张 GPU，每张约 93 GB
- **KV Cache**（128K 上下文时的峰值）：DSA 机制下约为标准 Attention 的 40-60%，估计约 8-12 GB/GPU
- **MoE 专家路由缓冲区**：约 4-6 GB/GPU（路由过程中的临时激活值）
- **框架开销**（CUDA context、推理框架内核）：约 2-3 GB/GPU
- **通信缓冲区**（TP 的 all-reduce 缓冲）：约 1-2 GB/GPU

单卡峰值显存需求 (FP8, 128K 上下文)：约 $93 + 12 + 6 + 3 + 2 = \text{约 } 116 \text{ GB}$ 。

这就是为什么即使使用 FP8，官方仍然要求 8 张 GPU 且设置 `--gpu-memory-utilization 0.85`——在 80GB H100 上，可用显存为 $80 \times 0.85 \approx 68 \text{ GB/卡}$ ，8 卡合计约 544 GB，减去权重 744 GB（跨卡分布后各卡约 93 GB）加上 KV Cache 等动态开销，刚好处于临界状态。

2.2 GPU 选型分析

2.2.1 候选 GPU 规格对比

2.2.2 显存容量约束分析

GLM-5.1 的部署核心约束是**总显存容量**，而非算力。以下是不同 GPU 配置下的可行性分析：

核心结论：对于 FP8 精度，部署 GLM-5.1 的显存门槛约为 **640 GB 总显存**（8×80 GB），这是 8 张 H100/A100 级别 GPU 的配置。消费级/专业可视化 GPU（24-48GB 级别）即使使用 8 卡也无法满足 FP8 部署需求。

表 5: 适用于 GLM-5.1 部署的 GPU 规格对比

GPU	显存	带宽	FP16 算力	互联	单卡价格
H200 SXM	141 GB	4.8 TB/s	990 TFLOPS	NVLink 900GB/s	~\$35K
H100 SXM	80 GB	3.35 TB/s	990 TFLOPS	NVLink 900GB/s	~\$28K
H100 PCIe	80 GB	2.0 TB/s	756 TFLOPS	无 NVLink	~\$24K
A100 SXM	80 GB	2.0 TB/s	312 TFLOPS	NVLink 600GB/s	~\$15K
A100 PCIe	80 GB	1.9 TB/s	312 TFLOPS	无 NVLink	~\$12K
A6000 Ada	48 GB	0.96 TB/s	91 TFLOPS	无	~\$6.8K
L40S	48 GB	0.86 TB/s	91 TFLOPS	无	~\$8K
RTX 6000 Ada	48 GB	0.96 TB/s	91 TFLOPS	无	~\$6.8K
RTX 4090	24 GB	1.0 TB/s	83 TFLOPS	无	~\$1.6K

表 6: 不同 GPU 配置的 GLM-5.1 部署可行性 (FP8 精度)

GPU	数量	总显存	权重分布	判断
H200 SXM	8	1,128 GB	93 GB/卡	充足, 可支持 BF16 全精度
H100 SXM	8	640 GB	93 GB/卡	可行 (官方推荐), FP8 精度
H100 PCIe	8	640 GB	93 GB/卡	可行但缺少 NVLink, TP 通信受 PCIe 瓶颈
A100 SXM	8	640 GB	93 GB/卡	可行但 decode 阶段带宽受限
A6000 Ada	8	384 GB	48 GB/卡	不可行 : 总显存低于 FP8 权重
RTX 4090	8	192 GB	24 GB/卡	不可行 : 总显存远低于门槛

2.3 量化与显存的精确对应关系

量化是降低 GLM-5.1 部署门槛的核心手段。以下基于社区已有的 GGUF 量化数据(以 unsloth/GLM-5.1-GGUF 为例):

表 7: GLM-5.1 GGUF 量化变体的文件大小与显存需求

量化级别	文件总大小	纯权重显存	推荐总显存	推荐 GPU
BF16 (原始)	~1,488 GB	1,488 GB	1,700+ GB	12×H200 / 不可单机
FP8 (官方)	~744 GB	744 GB	850+ GB	8×H100 / 8×A100
Q8_0	~744 GB	744 GB	850+ GB	8×H100 (GGUF 无 TP 优势)
UD-Q6_K	~558 GB	558 GB	650+ GB	8×H100 或 10×A100
UD-Q5_K_M	~465 GB	465 GB	550+ GB	8×A100 (80GB)
UD-Q4_K_M	~465 GB	465 GB	550+ GB	8×A100 或 12×A6000
UD-Q3_K_M	~372 GB	372 GB	450+ GB	8×A6000 或 4×H100+CPU
NVFP4	~437 GB	437 GB	520+ GB	8×A100 或 4×H200
MXFP4	~408 GB	408 GB	490+ GB	8×MI300X
UD-IQ4_XS	~372 GB	372 GB	450+ GB	8×A6000 (需 CPU offload)
UD-IQ3_XXS	~279 GB	279 GB	350+ GB	8×A6000 或 Mac Studio
UD-IQ2_XXS	~186 GB	186 GB	260+ GB	4×A6000 + CPU
MLX 4.8-bit	~420 GB	420 GB	450+ GB	Mac Pro 192GB×3 不现实
MLX 2.5-bit	~230 GB	230 GB	270+ GB	Mac Pro M2 Ultra 192GB

注意：以上”推荐总显存”包含了 KV Cache、框架开销和一定的安全余量(约 15-20%)。使用 llama.cpp 的 CPU offload 功能可以进一步降低 GPU 显存需求，但会带来显著的吞吐量下降。

3 部署方案详析

3.1 方案 A：企业级全精度部署 (BF16 + 8×H200/H100)

这是性能最完整、成本也最高的方案。适用于对模型质量有极致要求的企业级生产环境。

硬件配置：

- 8× NVIDIA H200 SXM (141 GB 每卡)，总显存 1,128 GB
- 或 8× H100 SXM (80 GB 每卡)，总显存 640 GB
- NVLink 全互联 (900 GB/s)，NVSwitch 拓扑
- 推荐使用 DGX H100 或同等规格的 SuperPOD 节点

部署命令 (vLLM):

```
vllm serve "zai-org/GLM-5.1" \
  --tensor-parallel-size 8 \
  --gpu-memory-utilization 0.85 \
  --max-model-len 131072 \
  --speculative-config '{"method":"mtp","num_speculative_tokens":3}' \
  --tool-call-parser glm47 \
  --reasoning-parser glm45
```

预期性能:

- 单请求 decode 速度: 40–60 tokens/s (H100 SXM, BF16)
- 并发能力: 5–10 个并发请求 (受 KV Cache 容量限制)
- Prefill 速度: 128K token Prompt $\approx$ 5–8 秒

优势: 全精度无质量损失, NVLink 保证 TP 通信效率, 支持长上下文。

劣势: 硬件成本极高, 功耗大 (8×H100 约 5,600W), 需要数据中心级散热和供电。

3.2 方案 B: 官方 FP8 量化部署 (8×H100/A100)

这是官方推荐的标准部署方案, 在精度与成本之间取得了最佳平衡。官方提供了预量化的 FP8 模型 (zai-org/GLM-5.1-FP8)。

硬件配置:

- 8× NVIDIA H100 SXM (80 GB), 总显存 640 GB
- 或 8× A100 SXM (80 GB), 总显存 640 GB
- 对于 H100: NVLink 全互联; 对于 A100: NVLink 600 GB/s

部署命令 (SGLang):

```
docker run --gpus all --shm-size 32g -p 30000:30000 \
  -v ~/.cache/huggingface:/root/.cache/huggingface \
  --env "HF_TOKEN=<your_token>" --ipc=host \
  lmsysorg/sglang:v0.5.11 \
  python3 -m sglang.launch_server \
    --model-path "zai-org/GLM-5.1-FP8" \
    --host 0.0.0.0 --port 30000 \
    --tp-size 8 \
    --mem-fraction-static 0.85 \
    --speculative-algorithm EAGLE \
    --speculative-num-steps 3 \
    --speculative-eagle-topk 1 \
    --speculative-num-draft-tokens 4
```

预期性能：

- Decode 速度：30–50 tokens/s (H100 SXM), 20–35 tokens/s (A100 SXM)
- 并发能力：8–15 个并发请求
- Prefill 速度：128K token $\approx$ 4–6 秒 (H100), 6–10 秒 (A100)

优势：官方支持，FP8 质量损失可忽略 ($< 0.5\%$ 困惑度差异)，社区最成熟的部署路径。

劣势：仍需 8 卡企业级 GPU，硬件门槛高。

3.3 方案 C：GGUF 量化方案 (Q4_K_M + llama.cpp/Ollama)

对于没有 8 卡企业级 GPU 的团队，GGUF 量化是降低硬件门槛的主要途径。

硬件配置选项：**C-1：8×A100 全 GPU 模式**

- GGUF Q4_K_M 文件大小 ~ 465 GB，8×80 GB 全 GPU 加载
- llama.cpp CUDA backend，tensor-parallel 支持
- 预期速度：15–25 tokens/s

C-2：4×A100/A6000 + CPU offload

- 4 张 GPU 承载最活跃的 MoE 专家层 (约 60% 权重)
- 剩余 40% 权重 offload 到系统内存 (DDR5 512+ GB)
- 预期速度：5–10 tokens/s (受 CPU-GPU 数据传输瓶颈)

C-3：纯 CPU 模式 (仅概念验证)

- 需要 ~ 500 GB 以上系统内存 (如双路 EPYC + 768 GB DDR5)
- Q4_K_M 纯 CPU 推理
- 预期速度：0.5–2 tokens/s (仅可用于验证模型是否加载成功，不可用于实际使用)

部署命令 (llama.cpp with CUDA)：

```
# 下载 Q4_K_M GGUF 文件 (以 unsloth 为例)
# 来自 https://huggingface.co/unsloth/GLM-5.1-GGUF

# 多 GPU 推理 (4卡示例, 其余 offload 到 CPU)
./llama-cli \
  -m GLM-5.1-UD-Q4_K_M-00001-of-00011.gguf \
  -ngl 60 \
  -c 32768 \
  -fa 1 \
  --tensor-split 24,24,24,24
```

说明：`-ngl 60` 表示将前 60 层 (MoE 中激活频率最高的部分) 加载到 GPU，其余层留在 CPU。`--tensor-split` 指定每张 GPU 的显存配额 (单位 GB)。对于 GLM-5.1 这样的 MoE 模型，GGUF 的 CPU offload 性能取决于 MoE 路由的频率分布——如果大部分 token 都路由到 GPU 驻留的专家，则性能能接近纯 GPU；否则会频繁触发 CPU-GPU 数据搬运。

3.4 方案 D：NVFP4 / MXFP4 新精度格式

这是 NVIDIA 和 AMD 分别推出的 4-bit 浮点格式，相比传统 INT4 量化有更好的动态范围和精度保持。

D-1: NVIDIA NVFP4 (Blackwell 架构专用)

NVFP4 是 NVIDIA Blackwell (B100/B200) 架构引入的原生 4-bit 浮点格式。与 INT4 不同，NVFP4 保留了浮点的指数位，在极低精度下仍能覆盖较大的数值范围。

- 模型大小：~437 GB（社区已有 lukealonso/GLM-5.1-NVFP4）
- 需 Blackwell 架构 GPU (B100/B200)，利用硬件原生 FP4 Tensor Core
- 8×B100 或 4×B200 (B200 显存更大)
- 理论吞吐量可达 FP8 的 2×（受益于 4-bit 内存带宽减半）

D-2: AMD MXFP4 (MI300X/MI350 专用)

AMD 的 MXFP4 是面向 ROCm 生态的 4-bit 浮点格式。

- 模型大小：~408 GB（社区已有 amd/GLM-5.1-MXFP4）
- 需 AMD Instinct MI300X (192 GB) 或更新型号
- 4×MI300X（共 768 GB）即可承载，且有较大 KV Cache 余量

3.5 方案 E：Apple Silicon 方案 (MLX 量化)

Apple Silicon 的统一内存架构使其在超大模型推理上有独特优势，但 GLM-5.1 的体量带来严峻挑战。

表 8: Apple Silicon 运行 GLM-5.1 的可行性分析

设备	最大统一内存	内存带宽	可跑量化	预期速度
Mac Studio M2 Ultra	192 GB	800 GB/s	MLX 2.5-bit	2–4 tok/s
Mac Pro M2 Ultra	192 GB	800 GB/s	MLX 2.5-bit	2–4 tok/s
Mac Studio M4 Max	128 GB	546 GB/s	不可行	—
MacBook Pro M4 Max	128 GB	546 GB/s	不可行	—

关键限制：

- 即使用 MLX 2.5-bit 量化（~230 GB），也需要 192 GB 统一内存的设备，且几乎没有 KV Cache 余量（仅能运行极短上下文）
- Mac Pro M2 Ultra 是唯一“勉强可跑”的 Apple 设备，但速度（2–4 tokens/s）无法满足实际使用
- Apple Silicon 不支持 CUDA，MLX 对 GLM-5.1 的 MoE 架构优化尚不成熟
- 结论：Apple Silicon 不适合 GLM-5.1 的本地部署，不推荐此方案

3.6 方案 F：云服务方案

对于缺少硬件预算的团队，云服务是最快上手的方式。

F-1: Together AI API (推荐)

Together AI 是 GLM-5.1 的官方推理服务商之一。

- 提供 OpenAI 兼容的 Chat Completions API
- 无需管理基础设施，零部署成本
- 价格：按 token 计费（具体价格以官方页面为准，参考同类 700B+ 模型约 \$3–8/1M input tokens, \$8–15/1M output tokens）

F-2: HuggingFace Inference Endpoint

通过 HuggingFace 的推理端点直接部署 GLM-5.1。

- 按 GPU 实例小时计费
- 以 8×H100 实例为例，约 \$25–40/小时
- 适合间歇性使用、批量评估、或作为 PoC 阶段的过渡方案

F-3: 算力租赁 (RunPod / Vast.ai / AutoDL)

表 9: 主流算力租赁平台对比（2026 年参考价格）

平台	H100 80GB/时	A100 80GB/时	备注
RunPod	\$2.49–3.49	\$1.49–2.29	按需和竞价实例
Vast.ai	\$1.80–3.00	\$1.00–1.80	社区托管，价格波动大
AutoDL（国内）	\$2.00–3.50	\$1.20–2.00	国内部署首选，中文支持好
Lambda Labs	\$2.49–2.99	\$1.49–1.99	北美地区

F-4: HuggingChat (免费)

GLM-5.1 已直接部署于 HuggingChat (huggingface.co/chat)，可免费使用，适合快速体验模型能力，无需任何部署。

4 推理框架选型

4.1 框架能力矩阵

4.2 vLLM (官方首推)

vLLM 是目前生产环境部署 GLM-5.1 的首选框架。其核心优势在于：

PagedAttention：将 KV Cache 按页（page）管理，类似操作系统的虚拟内存。这避免了传统框架中的显存碎片化问题，允许更高效的 batch 调度，将显存利用率从传统框架的 30–50% 提升至 80–90%。

Continuous Batching：动态地将新请求插入正在进行的 batch 中，无需等待整个 batch 完成，大幅提升吞吐量。

MTP (Multi-Token Prediction)：GLM-5.1 在 vLLM 中支持 MTP 推测解码，每次可预测 3 个 token，将 decode 吞吐量提升 30–50%。

生产特性：OpenAI 兼容 API、Prometheus 监控指标、量化支持（FP8/INT8/AWQ）、前缀缓存（Prefix Caching）。

表 10: GLM-5.1 支持的推理框架对比

框架	最低版本	核心优势	适用场景	成熟度
vLLM	v0.19.0+	生产级吞吐优化, PagedAttention	高并发在线服务	成熟
SGLang	v0.5.10+	RadixAttention, 编译优化	低延迟服务	成熟
llama.cpp	最新版	GGUF 量化, CPU-GPU 混合	个人/小团队	成熟
Ollama	最新版	一键部署, 自动下载	个人快速体验	成熟
KTransformers	v0.5.3+	异构计算 (GPU+CPU+NPU)	降低硬件门槛	实验性
xLLM	v0.8.0+	华为昇腾硬件	国产化替代	发展中
Transformers	v0.5.3+	HuggingFace 原生	研究/原型验证	基准

4.3 SGLang

SGLang 是来自 LMSys 的新一代推理框架，在延迟敏感场景下表现优异。

RadixAttention: 基于前缀树 (Radix Tree) 的 KV Cache 复用。当多个请求共享相同的 system prompt 或文档前缀时，RadixAttention 自动共享 KV Cache，避免重复计算。对于 Agent 场景（每次工具调用后重新发送完整历史），这一特性尤为关键。

EAGLE 推测解码: 比 MTP 更激进的推测解码策略，GLM-5.1 在 SGLang 上使用 EAGLE (3 步, topk=1, 4 个草稿 token)，在兼容性上优于 MTP。

Docker 一键部署: SGLang 提供了完整的 Docker 镜像，开箱即用。

4.4 llama.cpp / Ollama

作为本地 LLM 推理的基石，llama.cpp 在 GLM-5.1 的 GGUF 量化方案中不可替代。

关键能力:

- **CPU-GPU 混合推理**: 可将部分层 offload 到系统内存，大幅降低 GPU 需求
- **跨平台**: 支持 CUDA、Metal (Apple Silicon)、Vulkan、ROCm
- **任意量化级别**: 从 IQ1_M (~1.7 bpw) 到 Q8_0 (8 bpw) 的完整光谱
- **上下文压缩**: 支持 Flash Attention，降低长上下文的 KV Cache 开销

Ollama 在 llama.cpp 之上封装了类 Docker 的使用体验，但对于 GLM-5.1 这样的超大规模模型，Ollama 的默认配置可能不足，建议直接使用 llama.cpp 进行精细调参。

4.5 KTransformers (异构计算方案)

KTransformers 由 kvcache-ai 团队开发，专注于利用异构计算资源降低超大模型的部署门槛。

核心思路: 将 MoE 模型中激活频率低的专家层 offload 到 CPU 甚至是 NPU，仅在 GPU 上保留高频激活的专家和 Attention 层。对于 GLM-5.1 (激活比仅 5.4%)，大部分专家在特定任务下几乎不会被激活，因此 CPU offload 的实际影响远小于理论上的“跨 PCIe 传输全部权重”。

实测潜力: KTransformers 宣称可以在 4×RTX 4090 (24 GB × 4) 的配置上运行 DeepSeek-V3 (671B MoE)，实现 10–15 tokens/s 的可用速度。对于 GLM-5.1 的类似架构，理论上也可以在 4×A6000 (48 GB × 4 = 192 GB) 上实现可用部署。但这一方案尚处实验阶段，生产稳定性有待验证。

4.6 xLLM（华为昇腾方案）

对于有信创/国产化需求的团队，xLLM 是基于华为昇腾（Ascend）硬件的推理框架。

- 需 Ascend 910B 或更新型号
- 生态远不如 CUDA 成熟，模型适配需额外工作量
- 适合信创合规要求下的国企/政府场景

5 成本预估

5.1 硬件采购成本

表 11: GLM-5.1 各部署方案的硬件采购成本估算（2026 年 5 月参考价）

方案	GPU 配置	GPU 总价	服务器总价	总预算
A: 全精度	8×H200 SXM	\$280K	\$80–120K	\$360–400K
A: 全精度	8×H100 SXM	\$224K	\$70–100K	\$294–324K
B: FP8 标准	8×H100 SXM	\$224K	\$70–100K	\$294–324K
B: FP8 标准	8×A100 SXM	\$120K	\$50–80K	\$170–200K
C: Q4 GGUF	8×A100 PCIe	\$96K	\$50–70K	\$146–166K
C: Q4 GGUF	4×A6000+	\$27K	\$20–30K	\$47–57K
D: NVFP4	4×B200	\$140K	\$60–80K	\$200–220K
D: MXFP4	4×MI300X	\$60K	\$40–60K	\$100–120K

说明：服务器总价包括双路服务器平台（CPU、内存 512 GB+、NVMe 存储、网络、散热、电源等）。GPU 价格为 2026 年 5 月大致的市场参考价，实际采购（尤其是批量采购）可能有 10–20% 的折扣。

5.2 云服务成本对比

5.2.1 按需实例 TCO 分析

以 8×H100 的 HuggingFace Inference Endpoint 为例：

- **时租成本：**约 \$30/小时（含 GPU 实例 + HuggingFace 平台溢价）
- **月租成本**（720 小时/月）： $30 \times 720 = \$21,600$ /月
- **年租成本：**约 \$259,200/年（无预留折扣）
- **3 年 TCO：**约 \$777,600

5.2.2 API 调用成本

以 Together AI API 的典型 GLM-5.1 定价为例（参考同级别 MoE 模型）：

- **Input tokens:** 约 \$3-6 / 百万 tokens
- **Output tokens:** 约 \$8-15 / 百万 tokens
- **按典型 Agent 场景估算：** 单次 Agent 任务平均消耗 50K input + 10K output tokens，成本约 \$0.15-0.45/次
- **如果每天 100 次 Agent 调用：** \$15-45/天，\$450-1,350/月
- **如果每天 1,000 次 Agent 调用：** \$150-450/天，\$4,500-13,500/月

5.2.3 自有硬件 vs 云服务的盈亏平衡点

表 12: 自有硬件 vs 云服务/API 的 3 年 TCO 对比（单位：万美元）

方案	初始投入	年运维	3 年 TCO	适用场景
自购 8×H100	\$30	\$3	\$39	7×24 满负荷使用
自购 8×A100	\$18	\$2.5	\$25.5	中等负载使用
云 GPU 按时租	\$0	\$12-22/年	\$36-66	间歇性使用 (50%)
API 按量计费	\$0	\$5-16/年	\$15-48	低频 Agent 场景

粗略决策规则：

- 每天使用少于 8 小时 → 云 GPU 按时租更划算
- 每天使用超过 16 小时、负载可预测 → 自购硬件更划算
- 低频 Agent 调用（每天 < 500 次）→ API 按量计费最经济
- 高频调用 + 对延迟敏感 + 涉及敏感数据 → 必须自购硬件

5.3 电力与运维成本

- 8×H100 服务器功耗：约 5,600-7,000W（含 CPU、内存、散热）
- 年电力成本（按 \$0.12/kWh）：7 kW × 8,760 h × \$0.12 ≈ \$7,360/年
- 数据中心托管（8U 机柜 + 电力 + 网络）：约 \$1,500-2,500/月
- 运维人力：通常由现有工程团队兼任，边际成本低

6 性能预期与实测分析

6.1 各方案理论吞吐量估算

Decode 阶段的理论吞吐量可以通过“显存带宽 ÷ 模型读取量”公式估算。对于 MoE 模型，每次生成一个 token 不需要读取全部 744B 权重（仅激活 40B），但 KV Cache 的读取仍依赖于上下文长度。

表 13: 各方案的预期 Decode 速度估算（单请求，32K 上下文）

方案	GPU	每 token 数 据读取量	有效带宽	估算速度
A: BF16	8×H100	~90 GB	8×3.35 TB/s	40–60 tok/s
B: FP8	8×H100	~48 GB	8×3.35 TB/s	45–65 tok/s
B: FP8	8×A100	~48 GB	8×2.0 TB/s	30–45 tok/s
C: Q4 + 4×A6000	4×A6000	~35 GB	4×0.96 + CPU	5–10 tok/s

6.2 上下文长度对显存的影响

GLM-5.1 支持 128K 上下文，但实际可用上下文受 GPU 显存约束。

表 14: 不同上下文长度下 KV Cache 的额外显存占用（FP8, 8×H100）

上下文	KV Cache/GPU	权重 +KV+ 开销	80GB 占用率	状态
4K	~1.5 GB	~98 GB	122%	溢出
8K	~3.0 GB	~99 GB	124%	溢出
32K	~6.0 GB	~102 GB	128%	溢出
128K	~12 GB	~108 GB	135%	严重溢出

这一计算揭示了一个残酷的现实：即使使用 FP8 精度、8×H100 的配置，在不开启 KV Cache 量化的前提下，单卡 80GB 显存仍然面临溢出风险。这就是官方在部署示例中强调 `--mem-fraction-static 0.85` 的原因——必须预留 15% 的显存给 KV Cache 和其他动态分配。

缓解措施：

- vLLM 的 FP8 KV Cache 量化：将 KV Cache 从 FP16 压缩到 FP8，减少约 50% 的 KV Cache 显存
- Prefix Caching：在 Agent 场景中，多轮对话共享 system prompt 和历史前缀，避免重复存储
- vLLM 的 Automatic Prefix Caching (APC)：自动识别和复用公共前缀

6.3 并发能力评估

表 15: 不同配置下的预估并发能力（FP8, 32K 上下文）

配置	可用总显存	单请求 KV Cache	理论并发	实际并发
8×H100 (FP8 量化)	~500 GB	~6 GB/req	~80	8–15
8×A100 (FP8 量化)	~500 GB	~6 GB/req	~80	5–10
4×A6000 (Q4 GGUF)	~160 GB	~6 GB/req	~25	2–3

说明：理论并发受 KV Cache 容量约束，实际并发还受 GPU 算力和 Decode 带宽约束。对于 Agent 场景（长会话、频繁工具调用），建议保守估计并发数。

7 推荐方案矩阵

7.1 按场景推荐

表 16: GLM-5.1 部署方案场景推荐矩阵

场景	推荐方案	预算范围	备注
个人研究者	API + HuggingChat	\$50–500/月	无需硬件，免费体验可用 HuggingChat
小型创业团队	云 GPU 按时租 + API 混合	\$500–5K/月	弹性伸缩，按需付费
中型研发团队	自购 8×A100 + FP8	\$18–25 万	一次性投入，3 年回本
企业级平台	自购 8×H100 + BF16/FP8	\$30–40 万	全精度、高并发、低延迟
信创合规	xLLM + 华为昇腾	视配置	国产化替代，生态受限

7.2 决策流程图

以下是一个简化的决策逻辑：

1. 是否有 \$20 万以上的硬件预算？
 - 是 → 方案 B（自购 8×A100/H100 + FP8），一步到位
 - 否 → 进入问题 2
2. 日调用量是否超过 500 次 Agent 任务？
 - 是 → 云 GPU 按时租（8×H100 约 \$30/时），弹性调配
 - 否 → 进入问题 3
3. 是否有数据安全/合规要求（不可出内网）？
 - 是 → 方案 C（GGUF Q4 + 4×A6000 + CPU offload），最低自建方案
 - 否 → API 按量计费（Together AI / HuggingFace），零运维

7.3 最终建议

个人总结

对于大多数研发团队，GLM-5.1 的部署建议如下：

1. **PoC/探索阶段**：先使用 HuggingChat（免费）或 Together AI API（< \$500/月）体验模型能力，确认是否满足业务需求。
2. **小规模使用阶段**：使用云 GPU 按时租（RunPod/Vast.ai 8×H100 ≈ \$25 – –30/时），每周运行 20–40 小时，月成本约 \$2,000–5,000。此时无需任何硬件投入。
3. **规模化阶段**（确认 GLM-5.1 为团队核心生产力工具）：采购 8×A100 SXM 服务器（约 \$18–20 万），部署官方 FP8 模型。3 年 TCO 对比云方案可节省 40–60%。
4. **注意**：当前 **GLM-5.1 不适合消费级硬件部署**。即使在极端量化（IQ2）下，仍需至少 4 张

专业 GPU 或顶配 Mac Pro，且速度仅勉强可用。对于个人开发者，建议直接使用云 API，或将目光转向更轻量但同样优秀的模型（如 Qwen3-Coder-32B、DeepSeek-Coder-V2）。

附录 A：关键数据速查

表 17: GLM-5.1 关键指标速查表

指标	数值
总参数量	744B
激活参数量	40B (5.4%)
原始精度	BF16 (~1,488 GB)
官方推荐精度	FP8 (~744 GB)
最低可用量化	Q4_K_M / IQ4_XS (~372-465 GB)
官方推荐 GPU	8× H100 SXM (80 GB)
最低 GPU 配置 (Q4 GGUF + CPU offload)	4× A6000 (48 GB) + 512 GB 系统内存
推理框架 (推荐)	vLLM ≥v0.19.0 / SGLang ≥v0.5.10
推理框架 (量化)	llama.cpp / KTransformers
API 服务商	Together AI, HuggingFace Inference
免费体验	HuggingChat (chat.z.ai)
开源协议	MIT / Apache 2.0

附录 B：社区量化资源链接

- 官方模型: [zai-org/GLM-5.1](#) (BF16) | [zai-org/GLM-5.1-FP8](#)
- GGUF 量化 (推荐): [unsloth/GLM-5.1-GGUF](#) | [bartowski/GLM-5.1-GGUF](#)
- NVFP4: [lukealonso/GLM-5.1-NVFP4](#)
- MXFP4: [amd/GLM-5.1-MXFP4](#)
- MLX (Apple Silicon): [mlx-community/GLM-5.1-DQ4plus-q8](#)
- AWQ 4-bit: [cyankiwi/GLM-5.1-AWQ-4bit](#)
- REAP 剪枝 + 量化: [0xSero/GLM-5.1-REAP-NVFP4](#)
- 官方 GitHub: [github.com/zai-org/GLM-5](#)
- 技术报告: [arXiv:2602.15763](#)